IBM i (AS/400) Authorized Pentest Checklist

WebTerm/3270 AS/400 security toolset + the Pentesting the AS/400 audit order -- authorized engagements and training only

Prepared by Britley Hoff | October 12, 2026

Combines the WebTerm/3270 AS/400 security toolset (Bridge_server/public/js/as400sec.js, mock-lpar/mock-as400.js) with the audit order from Pentesting the AS/400 and the operational-judgment scenarios in the Mainframe 101-105 and 200-series learning books. For authorized engagements and training against the bundled mock IBM i LPAR; every item names the real command or button it runs.

How to use this checklist: one item per step, in order. Check the box when the step is done, write what you found on the Notes line before moving on, even if the finding is "ran clean" or "n/a" - a row with no finding is still a row worth a note. Phases 5 and 7 interlock with earlier phases; don't skip ahead.

Phase 0 -- Authorization and Scoping

No tool in this phase. Nothing past this point runs without a signed ROE.

- [ ] Signed ROE on file, naming which LPAR, which IBM i release, and the library/object boundary (if any).

Notes: .....

- [ ] ROE states explicitly whether read-only enumeration is authorized versus anything that touches a production object.

Notes: .....

- [ ] Default/shared-credential enumeration is named in the ROE as its own authorized activity, not assumed to be covered by general scope.

Notes: .....

- [ ] Change-window and rollback language confirmed -- IBM i shops often run with less redundancy than assumed.

Notes: .....

- [ ] Data-handling clause covers screen captures: a Sign On banner or a profile list can itself contain production data.

Notes: .....

- [ ] Client sign-off obtained before the first command runs.

Notes: .....

Phase 1 -- Pre-Authentication Recon

Everything here happens before a password is successfully typed.

- [ ] Read the Sign On screen as a disclosure surface -- system name, subsystem, program/library fields.

Notes: .....

- [ ] Record system name, subsystem, and display device id exactly as shown.

Notes: .....

- [ ] Run the Default Credential Audit before signing on for real. Security panel -> Default Credential Audit -> START.

Notes: .....

- [ ] Load any pre-engagement intel (userids from a kickoff call, a directory export) as additional candidates. Load file / Load from host.

Notes: .....

- [ ] Record every SUCCESS, every EXISTS (valid userid, no working password -- still a finding), and whether the run hit a LOCKOUT.

Notes: .....

Phase 2 -- Initial Authenticated Recon

Signed on. Before any drill-down tool, get the shop's overall posture and patch currency.

- [ ] Display network attributes. Command: DSPNETA -- ANALYZE NETWORK ATTRS.

Notes: .....

- [ ] Run the System Value Analyzer. Command: WRKSYSVAL -- ANALYZE SYSTEM VALUES.

Notes: .....

- [ ] Reason out which values should flag before reading the tool's verdict: QSECURITY below 40, QMAXSIGN at NOMAX, QPWDEXPITV at NOMAX, QALWOBJRST at ALL, QAUDCTL at NONE.

Notes: .....

- [ ] Run the PTF/CVE Currency Checker. Command: STRSQL (SYSTOOLS.CVE_INFO(), SYSTOOLS.GROUP_PTF_CURRENCY_LOCAL()) -- CHECK PTF/CVE CURRENCY.

Notes: .....

Phase 3 -- Identity and Privilege Enumeration

- [ ] Enumerate user profiles. Command: WRKUSRPRF -> DSPUSRPRF -- ENUMERATE PROFILES.

Notes: .....

- [ ] Flag special authorities, especially ALLOBJ/SECADM held together -- CRITICAL on their own.

Notes: .....

- [ ] Flag any privileged profile with LMTCPB(NO) -- the limitation compounds the risk, it doesn't just add to it.

Notes: .....

- [ ] Audit the IBM-supplied (Q) profile set. AUDIT SHIPPED PROFILES.

Notes: .....

- [ ] Confirm every shipped profile is non-interactive: STATUS(DISABLED) or PASSWORD(NONE).

Notes: .....

- [ ] Flag any default password on a shipped profile as CRITICAL, regardless of anything else true about it.

Notes: .....

- [ ] Flag a Q-named profile that is NOT on the real IBM-supplied list -- a classic "blend in with a shipped name" persistence technique.

Notes: .....

- [ ] Note the QSECOFR exception: a vaulted break-glass account is legitimate on its own; a default password on it is not.

Notes: .....

Phase 4 -- Object and Data Authority

- [ ] Scan object authority. Command: WRKOBJ / DSPOBJAUT -- SCAN OBJECT AUTHORITY.

Notes: .....

- [ ] Rate PUBLIC authority first: ALL critical, CHANGE high, USE low.

Notes: .....

- [ ] Check private grants on the object's own authority list -- a non-public user holding ALL/CHANGE on a sensitive object escalates the finding even when PUBLIC looks fine.

Notes: .....

- [ ] Confirm with the client which libraries/objects count as "sensitive" for this engagement (payroll, employee master, configuration) -- do not assume a generic pattern match.

Notes: .....

Phase 5 -- The Privilege-Escalation Triangle

Three scanners whose findings interlock. Run Phase 3's User Profile scan in the same sitting before Active Jobs -- the Active Job scanner cross-references its in-memory CRITICAL/HIGH list.

- [ ] Scan job descriptions. Command: WRKJOBD -- SCAN JOB DESCRIPTIONS.

Notes: .....

- [ ] Flag a job description whose USER() parameter lets a submitter run batch work under an identity they don't hold -- the platform's own setuid-binary equivalent.

Notes: .....

- [ ] Scan authorization lists. Command: WRKAUTL / DSPAUTL -- SCAN AUTH LISTS.

Notes: .....

- [ ] Treat one overbroad PUBLIC rating on an authorization list as a multiplier -- it covers every object the list secures, all at once.

Notes: .....

- [ ] Scan active jobs. Command: WRKACTJOB -- SCAN ACTIVE JOBS.

Notes: .....

- [ ] Cross-reference: is any profile flagged CRITICAL/HIGH in Phase 3 currently running a job? Weight that finding accordingly.

Notes: .....

Phase 6 -- The Network-Facing Surface

What can reach this box without going through a 5250 sign-on at all.

- [ ] Audit exit points / service registration. Command: WRKREGINF -- AUDIT EXIT POINTS.

Notes: .....

- [ ] Flag a missing or unregistered exit program on a sensitive exit point (FTP, remote command, remote SQL).

Notes: .....

- [ ] Audit NetServer (SMB) configuration. AUDIT NETSERVER CONFIG.

Notes: .....

- [ ] Check guest access and SMB signing requirements specifically.

Notes: .....

- [ ] Sweep IFS permissions. Command: WRKLNK -- SWEEP IFS PERMISSIONS.

Notes: .....

- [ ] Evaluate by owner and PUBLIC authority the same way as a native object -- under the hood, it is one.

Notes: .....

Phase 7 -- Adopted Authority and Live Bypass Testing

Authority leaking past a boundary the shop thinks it has drawn. The bypass probe actually drives screens and issues real commands -- confirm the ROE covers active testing, not just enumeration, before this phase.

- [ ] Scan the call stack for adopted authority. Command: WRKJOB OPTION(PGMSTK) -> DSPPGM -- SCAN CALL STACK.

Notes: .....

- [ ] Flag any program that adopts an owner's authority where the owner is a privileged profile.

Notes: .....

- [ ] Run the Menu/Command-Line Bypass Probe. PROBE FOR BYPASSES.

Notes: .....

- [ ] Confirm whether LMTCPB(YES) on the sign-on session actually reaches each stock utility's command line (WRKSPLF, WRKOUTQ, WRKJOBD, WRKUSRJOB, WRKACTJOB) -- the limit is checked at sign-on, not necessarily on every later screen.

Notes: .....

- [ ] Note which screens held the restriction and which quietly didn't.

Notes: .....

Phase 8 -- Reporting

- [ ] Export every tool's findings. Export CSV / JSON on each panel.

Notes: .....

- [ ] Write an executive summary a non-technical stakeholder can act on.

Notes: .....

- [ ] Rank findings by real exploitability, not one flattened severity scale.

Notes: .....

- [ ] Separate a narrow fix (disable one shipped profile) from a structural one (no process exists to review adopted-authority programs at all).

Notes: .....

- [ ] Time the report's release around the client's own change windows, not your own convenience.

Notes: .....

Where the Mock Stops

Every tool here was built and verified against WebTerm/3270's open-source mock IBM i LPAR, not a real customer system. The mock's shipped-profile list is the common base plus common-LP set; a real shop layers in whatever licensed products it has installed, each with its own service profiles to research, not assume. Exit-point findings here mean "nothing is positioned to intercept this," not "the actual exit program has a flaw" -- real exit programs are custom code the mock doesn't model the behavior of. Treat every item's command and risk language as a starting point for what to ask a real system, not a claim about what a real system will show.

Project reference: Bridge_server/DOCUMENTATION/webterm-security-tools-tutorial.md and webterm-security-tools-cheat-sheet.md cover this toolset and the rest of WebTerm/3270's security tooling (z/OS, z/VM, z/TPF) in more depth.